Mobility-Aware Assisted Deep Reinforcement Learning for Collaborative Task Migration and Resource Allocation in Vehicular Edge Computing

Yilin Yin, Peng Hou, *Member, IEEE*, Shihong Cui, Jin Huang, Jin Wang, *Member, IEEE*, Zhihui Lu, *Member, IEEE*, and Patrick C. K. Hung, *Senior Member, IEEE*

***Abstract*—Driven by advances in artificial intelligence and wireless communication technologies, vehicular edge computing (VEC) has become a vital component of intelligent transportation systems. However, high vehicle mobility in VEC may cause task execution interruptions as vehicles move beyond the coverage of the roadside units (RSUs). Additionally, uneven vehicle distribution and static RSUs deployment can lead to workload imbalance and inefficient resource allocation. Although collaborative task migration and resource allocation among RSUs can mitigate some existing challenges, vehicle mobility-aware proactive decision-making is the key to achieving lower delay and better workload balancing. Therefore, we propose a novel mobility-aware task migration (MATM) framework, which integrates a Mamba-based trajectory prediction module and a soft actor–critic (SAC)-based intelligent decision-making module. MATM leverages a multi-level residual connection architecture to accurately capture the future spatiotemporal information of vehicles. This information serves as environmental state guidance for the SAC decision module to perform dynamic task migration and resource allocation. By integrating spatiotemporal information mining with deep reinforcement learning (DRL), MATM enables real-time optimal forward-looking service decisions. Extensive experiments on real-world vehicle trajectory datasets from Cologne demonstrate that MATM outperforms both heuristic and DRL-based methods, reducing the average task completion delay by 8.3%–54.5%.**

***Index Terms*—Vehicular edge computing, edge intelligence, task migration, deep reinforcement learning, trajectory prediction.**

Received 23 September 2025; revised 2 December 2025; accepted 25 January 2026. Date of publication 3 February 2026; date of current version 20 July 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62072321, in part by the Science and Technology Program of Jiangsu Province under Grant BZ2024062, in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 22KJA520007, in part by the Suzhou Planning Project of Science and Technology under Grant 2023ss03, Grant SYG2025129, and Grant SNG2025010, and in part by the Key Laboratory of General Artificial Intelligence and Large Models in Provincial Universities, Soochow University. The review of this article was coordinated by Dr. Bartolomeo Montrucchio. *(Corresponding authors: Peng Hou; Jin Wang.)*

Yilin Yin, Peng Hou, and Jin Wang are with the School of Future Science and Engineering, Key Laboratory of General Artificial Intelligence and Large Models in Provincial Universities, Soochow University, Suzhou 215222, China (e-mail: 20245258017@stu.suda.edu.cn; penghou@suda.edu.cn; wjin1985@suda.edu.cn).

Shihong Cui is with the Marketing Department, Tianyijiaotong Technology Company Ltd., Suzhou 215000, China (e-mail: cuishihong202406@163.com).

Jin Huang is with the School of Vehicle and Mobility, Tsinghua University, Beijing 100084, China (e-mail: huangjin@tsinghua.edu.cn).

Zhihui Lu is with the College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200438, China (e-mail: lzh@fudan.edu.cn).

Patrick C. K. Hung is with the Faculty of Business and Information Technology, Ontario Tech University, Oshawa, ON L1G 0C5, Canada (e-mail: patrick.hung@uoit.ca).

Digital Object Identifier 10.1109/TVT.2026.3660321

I. Introduction

With the rapid development of artificial intelligence and wireless communication technologies, a growing number of applications such as augmented reality, virtual reality, and real-time navigation have been widely integrated into the internet of vehicles (IoV) [1]. The onboard computing capabilities of vehicles are typically insufficient to meet the requirements of these computation-intensive and delay-sensitive applications [2]. Moreover, local computation on vehicles elevates energy consumption and reduces driving range. As an illustration, an electric vehicle equipped with a 2 kW onboard computing system may experience up to a 25% reduction in driving range under high computational workload [3]. To address the above issues, vehicular edge computing (VEC) has been proposed as an effective solution [4], [5], [6], which allows vehicles to offload tasks to nearby roadside units (RSUs). Unlike traditional approaches relying on remote cloud computing, VEC significantly reduces task transmission distance and lowers processing delay. VEC mitigates network disruptions and enhances reliability [7].

Vehicle mobility is a key feature that differentiates VEC from conventional edge computing paradigms [8]. As a result, a vehicle may leave the coverage area of its associated RSUs before the offloaded task is completed and the results are fed back, which may lead to connection interruptions and task execution failures. In addition, due to the static deployment of RSUs and the highly uneven spatiotemporal distribution of vehicles, significant workload imbalance may arise among edge computing resources [9]. For example, during peak hours in commercial districts, a large number of vehicles may simultaneously offload tasks to the same RSU, causing severe communication congestion and potential overload, which significantly reduces the task success rate. Conversely, RSUs located on side streets or in areas with sparse traffic may remain underutilized due to the limited number of

0018-9545 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

nearby vehicles. Such workload imbalance decreases overall system efficiency and can exacerbate the overload of heavily burdened RSUs. Furthermore, the limited channel resources between vehicles and RSUs may lead to substantial channel interference, further increasing task transmission delay [10].

Collaboration among RSUs can effectively address issues caused by vehicle mobility and uneven resource utilization [11], [12]. On one hand, when a vehicle disconnects from its currently associated RSU, inter-RSU collaboration facilitates the forwarding and feedback transmission of intermediate task results. This ensures the successful delivery of computation outcomes to the vehicle. On the other hand, dynamic task migration between RSUs facilitates workload balancing across edge resources, reducing the risk of RSU overload or underutilization. At the same time, rational allocation of communication channels between vehicles and RSUs can effectively alleviate the uneven distribution of communication resources.

Due to the high-speed mobility of vehicles, inappropriate task migration and resource allocation decisions may result in increased task completion delay. For example, consider a vehicle in the city center offloading a task to an RSU under heavy load. The task needs to be migrated to an idle RSU. If the chosen RSU is located opposite to the vehicle's moving direction, the longer return path may increase result feedback delay [13], [14]. Therefore, task migration strategies should incorporate more accurate and efficient vehicle trajectory prediction. Existing trajectory prediction methods face a trade-off between efficiency and accuracy. Long short-term memory (LSTM) networks have a relatively simple architecture, which allows for fast computation, but they often fail to capture long-range dependencies in highly dynamic traffic scenarios, limiting prediction accuracy. Transformer-based models use self-attention mechanisms to better capture temporal and spatial dependencies, resulting in improved accuracy. However, their higher model complexity and the need to process long sequences lead to increased computational cost and lower efficiency, making them less suitable for delay-sensitive VEC environments. In contrast, the recently proposed Mamba architecture, built upon linear state space models (SSMs) [15], strikes a favorable balance between modeling capability and efficiency, demonstrating strong potential for real-time vehicular trajectory prediction [16], [17].

Meanwhile, due to the highly dynamic nature of VEC, joint optimization of task migration and resource allocation becomes particularly challenging. Moreover, the joint optimization problem is inherently non-stationary, making it difficult for traditional optimization algorithms to develop effective solutions. Owing to its capacity to make adaptive and dynamic decisions, deep reinforcement learning (DRL) has emerged as an effective tool for addressing complex task scheduling problems in dynamic VEC environments [18], [19]. Through collaborative communication, RSUs share observed information and vehicle mobility awareness data, enabling DRL to acquire a more comprehensive view of the environment and optimize policy learning and decisions.

Inspired by the above, we propose a mobility-aware task migration (MATM) framework that integrates a Mamba-based trajectory prediction module with a soft actor-critic (SAC) DRL decision-making module. The integration of spatiotemporal information extraction and intelligent decision-making in MATM enables more efficient task migration and resource allocation, which in turn leads to reduced task completion delay, improved success rate, and better workload balance. The main contributions of this paper are summarized as follows:

- We focus on a collaborative VEC scenario that incorporates vehicle mobility, RSU workload, and channel interference. We formulate the joint optimization of task offloading, migration, and channel selection as a mixed integer nonlinear programming (MINLP) problem, with the objective of minimizing the task completion delay.
- To predict future vehicle trajectory information for guiding task migration and feedback, we propose a multi-layer Mamba-based residual network that integrates multi-layer feature extraction with residual connections. The multi-layer structure captures intricate patterns and the residual connections enhance information flow, thus improving prediction accuracy and learning efficiency.
- By combining forward-looking trajectory prediction, we propose the MATM framework combined with the SAC-based intelligent decision-making module. MATM leverages the advantages of spatiotemporal information mining and intelligent decision-making, enabling the effective execution of optimal task migration and resource allocation decisions in VEC scenarios.
- We conduct extensive experiments using real-world vehicular trajectory data from the city of Cologne. Parameter sensitivity analysis and ablation studies are designed to evaluate the effectiveness of MATM. Experimental results demonstrate that MATM significantly reduces the average task completion delay by 8.3% to 54.5% compared to the baselines.

The rest of the paper is organized as follows. Section II reviews the related work. Section III presents the system model and problem formulation. In Section IV, we present the MATM approach. Section V details the experimental setup and analyzes the results. Finally, Section VI concludes the paper.

II. Related Work

A. Task Migration

Task migration serves as an effective approach for workload balancing and improving task success rates in VEC scenarios [8], [20], [21], [22], [23], [24]. Several studies have utilized traditional optimization techniques for task migration. P. Dai et al. [20] proposed migrating tasks from edge servers to the cloud to alleviate edge-side congestion, effectively mitigating overload issues. H. Wang et al. [21] proposed a task migration strategy based on game-theoretic principles, effectively minimizing computational overhead while improving resource utilization and task success rates. However, their dependence on traditional optimization methods restricts adaptability to highly dynamic VEC scenarios.

Some studies have employed DRL-based intelligent decision-making to perform task migration, demonstrating the effectiveness of edge intelligence in alleviating VEC challenges.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

J. Li et al. [22] studied task remigration to improve system flexibility, but assumed static users and overlooked the impact of vehicle mobility. W. Fan et al. [8] proposed a twin delayed deep deterministic policy gradient (TD3)-based RSU cooperation approach that effectively facilitates collaboration between RSUs and accounts for vehicle mobility, but their method only applies to vehicles entering the next RSU.

In addition, a major focus of recent research has been on load balancing and task completion delay optimization. Q. Chen et al. [23] employed proximal policy optimization (PPO) for load-balancing task migration, while ignoring task feedback delay caused by vehicle mobility. C. Liu et al. [24] proposed a distributed counterfactual multi-agent DRL approach that improves Quality of Service (QoS) and reduces task completion delay, but ignores proactive migration for load balancing.

Therefore, a critical research gap persists in jointly achieving efficient load balancing via intelligent decision-making while proactively mitigating the impact of vehicle mobility on result feedback delay.

B. Mobility Awareness

Vehicle mobility significantly impacts the effectiveness of task offloading and migration. Therefore, accurately predicting vehicle mobility can guide task offloading and migration decisions, thereby improving overall QoS [25], [26], [27], [28], [29], [30].

Several studies have employed heuristic methods to predict vehicle positions to assist task offloading and migration decisions. Y. Li et al. [25] estimated whether tasks could be completed before vehicles leave the coverage area of the current RSUs, which supports the perception of vehicle mobility patterns and assists service scheduling. I. Maity et al. [26] designed a mobility management module based on a Markov predictor to forecast vehicle positions and mitigate the problem of insufficient RSU coverage. Zhao et al. [27] employed the extended Kalman filter to predict vehicle trajectories, acting as supplementary input for the PPO-based task offloading algorithm. However, these heuristic methods typically rely on simplistic model assumptions, limiting their ability to perform precise modeling in complex, dynamic VEC scenarios. As a result, they cannot provide the continuous, low-latency, multi-step forecasts necessary for effective task migration and result feedback delay management.

Some studies have already employed deep learning–based methods to predict vehicle mobility. J. Chen et al. [28] developed a trajectory prediction model based on LSTM to anticipate future vehicle positions and facilitate task migration. C.-L. Wu et al. [29] presented a Seq2Seq-based approach to enhance the precision of task migration. X. Zhang et al. [30] adopted a Transformer architecture for efficient trajectory prediction. However, the aforementioned deep learning-based methods fail to strike a favorable balance between prediction accuracy and computational efficiency.

Recently, Mamba has shown significant potential in trajectory prediction. Y. Huang et al. [31] proposed a Mamba architecture with a self-attention mechanism that reduces inference time and improves prediction accuracy in intersection trajectory prediction. S. Cheng et al. [32] combined the Mamba architecture with convolutional neural networks to reduce computational complexity for pedestrian trajectory prediction. Z. Xiao et al. [33] employed a Mamba-based approach utilizing temporal audio features to enhance uncrewed aerial vehicle trajectory estimation and classification, thereby improving prediction accuracy and effectiveness. Through the analysis of the above studies, Mamba is a promising trajectory prediction method, achieving a balanced trade-off between efficiency and accuracy. Building on these insights, we propose an innovative application of Mamba-based trajectory prediction to VEC scenarios. To better fit the characteristics of VEC, we further design a multi-level residual Mamba architecture, enabling efficient, low-delay trajectory prediction while enhancing adaptability to delay-sensitive VEC scenarios.

TABLE I
COMPARISON BETWEEN OUR WORK AND RELATED STUDIES

Work	Task migration	Workload balancing	Interference management	Trajectory prediction	Result feedback	Edge intelligence
Our	✓	✓	✓	✓	✓	✓
[20]	✓	✓	×	×	×	×
[21]	✓	×	✓	×	✓	×
[22]	✓	×	×	×	×	✓
[8]	✓	×	✓	×	✓	✓
[23]	✓	✓	✓	×	×	✓
[24]	✓	×	×	×	×	✓
[25]	×	×	✓	✓	×	×
[26]	×	×	×	✓	×	×
[27]	✓	×	×	✓	×	✓
[28]	✓	✓	×	✓	✓	✓
[29]	✓	×	×	✓	×	✓
[30]	✓	✓	×	✓	×	✓

Table I summarizes the key distinctions between our work and existing studies. 1) We adopt a novel framework that integrates trajectory prediction with intelligent decision-making to optimize task migration and resource allocation. 2) We jointly consider the impact of task migration on RSU workload and the effect of mobility-aware perception on result feedback delay. 3) We innovatively apply the Mamba-based trajectory predictions to the VEC task offloading scenarios.

III. System Model and Problem Formulation

A. Network Model

As shown in Fig. 1, we consider a VEC system involving multiple vehicles and RSUs. The system consists of R RSUs and V vehicles, represented by the sets $\mathcal{R} = \{1, 2, \ldots, R\}$ and $\mathcal{V} = \{1, 2, \ldots, V\}$, respectively. Each vehicle has limited resources and can only handle small tasks locally. When demand increases, tasks are offloaded to RSUs with greater computational capacity for processing [2]. Each RSU is equipped with a single antenna for wireless communication with vehicles, enabling the provision of computing services [5]. Communication between vehicles and RSUs is facilitated through vehicle-to-RSU (V2R), using the PC5 interface for task offloading and result feedback [23]. Furthermore, RSUs are interconnected through RSU-to-RSU (R2R) links, typically utilizing wired networks to ensure fast and reliable communication between RSUs [8], [23].

The system runs in discrete time slots $\mathcal{T} = \{1, 2, \ldots, T_{\max}\}$. Each time slot lasts for a duration denoted by Δt[1]. We assume

[1]We set Δt small to simplify sensing and stabilize channels, while ensuring sufficient state variation for effective DRL decisions. [14].

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

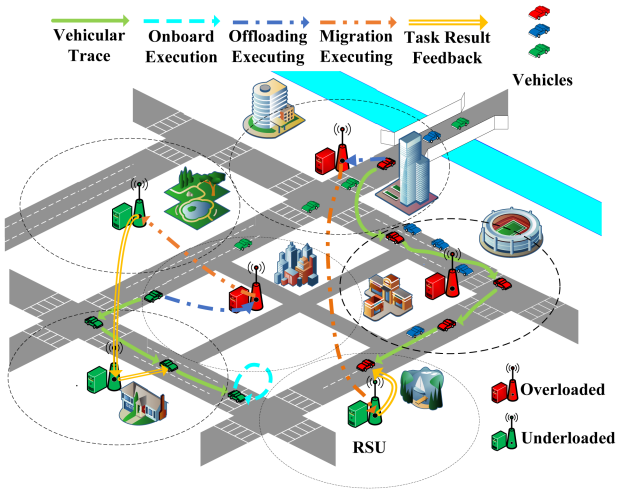


Fig. 1. Overview of collaborative RSUs in VEC scenario.

fixed positions for RSUs and dynamic positions for vehicles over time. The road surface is flat [23], and we ignore the height difference between the RSU and the vehicle. The positions of vehicle i and RSU j are denoted as $L_i^v = (x_i, y_i), \forall i \in \mathcal{V}$, $L_j^r = (x_j, y_j), \forall j \in \mathcal{R}$, respectively. We assume that each vehicle acquires its current position via GPS and broadcasts this information to the serving RSU, while RSUs share the received vehicle location data with each other [27].

The distance between vehicle i and RSU j can be expressed as

$$D_{i,j} = ||L_i^v - L_j^r||_2, \forall i \in \mathcal{V}, j \in \mathcal{R}. \tag{1}$$

The maximum communication radius of RSU $j \in \mathcal{R}$ is denoted as $\mathcal{D}_j$. If the distance between vehicle i and RSU j is less than $\mathcal{D}_j$, a service link is established between vehicle i and RSU j. Therefore, we can use the variable $C_{i,j} \in \{0, 1\}$ to represent whether vehicle i is connected to RSU j,

$$C_{i,j} = \begin{cases} 1, & \text{if } D_{i,j} \leq \mathcal{D}_j, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

In particular, vehicle i may be located within the coverage areas of multiple RSUs, but its task can only be connected to one RSU [9], i.e.,

$$\sum_{j \in \mathcal{R}} C_{i,j} \leq 1, \forall i \in \mathcal{V}, j \in \mathcal{R}. \tag{3}$$

At each time slot, vehicle i randomly generates one computation task $\Omega_i = \{e_i, c_i, \tau_i\}$, where e_i is the data size, c_i represents the number of CPU cycles required to process one bit of data, τ_i represents the allowable maximum delay. If the task completion time exceeds τ_i, the task will be dropped [30].

B. Task Migration Model

In the RSU collaborative VEC scenario, we assume that the vehicular task processing involves two decision-making steps.

1) Binary Task Offloading: If vehicle i is connected to RSU $j \in \mathcal{R}$, vehicle i decides whether to process its task locally or offload it to RSU j. This decision is represented by $\alpha_i \in \{0, 1\}$, where $\alpha_i = 1$ indicates that the task of vehicle i is offloaded to RSU j for execution. However, the vehicle i can only offload tasks to the RSU it is currently connected to, i.e.,

$$\sum_{j \in \mathcal{R}} \alpha_i C_{i,j} \leq 1, \forall i \in \mathcal{V}, j \in \mathcal{R}. \tag{4}$$

2) Binary Task Migration: After offloading, the task Ω_i may be migrated from its offloading RSU j to another RSU $k \in \mathcal{R}$, based on vehicle location and RSU workload. The migration decision variable can be defined as $\beta_{i,j,k} \in \{0, 1\}$, where $\beta_{i,j,k} = 1$ indicates that the task of vehicle i is migrated from the RSU j to RSU k. Specifically, when $k = j$, the task is migrated from RSU j to itself, meaning no migration occurs. The task Ω_i can only be migrated to a single RSU, i.e.,

$$\sum_{j \in \mathcal{R}} \sum_{k \in \mathcal{R}} \alpha_i C_{i,j} \beta_{i,j,k} \leq 1, \forall i \in \mathcal{V}, j, k \in \mathcal{R}. \tag{5}$$

C. Communication Model

1) V2R Uplink Communication: We assume that RSUs communicate with vehicles using non-orthogonal multiple access (NOMA)[2] [14], [25]. It allows multiple vehicles to transmit simultaneously in the same time slot and frequency, improving system capacity and spectrum utilization. To manage spectrum allocation, each RSU partitions its total bandwidth W into U subchannels, each with bandwidth W/U. Based on NOMA technology, vehicles sharing the same subchannel in an RSU form a NOMA cluster, where the RSU mitigates co-channel interference within the cluster via successive interference cancellation (SIC) [25]. The subchannels between vehicles and RSUs are represented by the set $\mathcal{U} = \{1, 2, \ldots, U\}$.

The channel power gain of the V2R uplink from vehicle $i \in \mathcal{V}$ to the RSU $j \in \mathcal{R}$ is expressed as [34]

$$H_{i,j}^{up} = \frac{G^{tran} G^{rece} \lambda^2}{(4\pi)^2 |D_{i,j}|^2} = \frac{g_0}{|D_{i,j}|^2}, \tag{6}$$

where G^{tran} and G^{rece} denote the antenna gains of the transmitter on vehicle i and the receiver on RSU j. λ is the signal wavelength. g_0 is the channel gain at reference distance 1m.

Let $Z_{i,j,u} \in \{0, 1\}$ denote the vehicle i communicating with the RSU j through subchannel $u \in \mathcal{U}$. Each vehicle can only select one subchannel to communicate with an RSU, i.e.,

$$\sum_{u=1}^{U} Z_{i,j,u} \leq 1, \forall i \in \mathcal{V}, j \in \mathcal{R}, u \in \mathcal{U}. \tag{7}$$

We assume the channel power gains are sorted in ascending order within each cluster [19]. Therefore, the interference for vehicle i communicating with the RSU j can be defined as

$$I_{i,j} = \sum_{u=1}^{U} \sum_{n=1}^{i-1} Z_{n,j,u} Z_{i,j,u} P_n H_{n,j}^{up}, \tag{8}$$

where P_n is the transmission power of vehicle $n \in \mathcal{V}$. According to Shannon's second theorem, the data transmission rate between

[2]NOMA is used to ensure feasible data transmission when multiple vehicles share the same RSU and subchannel [14], [25], rather than as a research focus.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

vehicle i and RSU j can be calculated as [14]:

$$R_{i,j}^{up} = \frac{W}{U}\log_2\left(1+\frac{P_i H_{i,j}^{up}}{I_{i,j}+\sigma_0^2}\right), \tag{9}$$

where σ_0^2 is the Gaussian noise. Thus, the uplink transmission delay from vehicle i to RSU j can be expressed as

$$T_{i,j}^{up} = \frac{\alpha_i C_{i,j} e_i}{R_{i,j}^{up}}. \tag{10}$$

2) R2R Communication: We assume that RSUs are connected via a wired network with communication capacity. Due to deployment constraints and physical limitations, the connections between RSUs are not fully meshed, and communication is achieved through multi-hop transmissions [30]. Therefore, the delay for task migration from RSU j to RSU k includes the propagation delay and the transmission delay, i.e.,

$$T_{i,j,k}^{mig} = \alpha_i C_{i,j}\beta_{i,j,k}\left(\frac{D_{j,k}}{\varrho}+\frac{Hops(j,k)e_i}{\Theta}\right), \tag{11}$$

where ϱ is the propagation rate of electromagnetic waves, Θ represents the R2R communication transmission rate and $Hops(j,k)$ denotes the number of hops in the shortest path between RSU j and k [30].

After the task computation is completed at RSU $k \in \mathcal{R}$, the task result may still need to be transmitted to the RSU $l \in \mathcal{R}$ where the vehicle is located via R2R communication. We define the variable $C'_{i,l} \in \{0,1\}$ to represent whether vehicle i is connected to RSU l when the task result is ready for feedback. When vehicle i is connected to RSU l, $C'_{i,l}=1$; otherwise, $C'_{i,l}=0$. Therefore, the task transmission delay from RSU k to RSU l via R2R communication is given by

$$T_{i,j,k,l}^{fb} = \alpha_i C_{i,j}\beta_{i,j,k}C'_{i,l}\left(\frac{D_{k,l}}{\varrho}+\frac{Hops(k,l)\mu_i e_i}{\Theta}\right), \tag{12}$$

where μ_i is the ratio of result/input data size of task Ω_i.

3) V2R Downlink Communication: The V2R downlink communication rate from RSU $l \in \mathcal{R}$ to vehicle $i \in \mathcal{V}$ is denoted as $R_{l,i}^{\text{down}}$. It can be derived from the V2R uplink channel model due to the symmetry of V2R uplink and downlink channels[3] [8]. Thus, the result feedback delay from the RSU l to vehicle i is defined as

$$T_{i,j,k,l}^{down} = \frac{\alpha_i C_{i,j}\beta_{i,j,k}C'_{i,l}\mu_i e_i}{R_{l,i}^{down}}. \tag{13}$$

D. Computation Model

1) Onboard Computation: We assume that the computing capability of vehicle $i \in \mathcal{V}$ is f_i. Then, the Onboard computation delay of the task of vehicle i can be expressed as

$$T_i^{loc} = \frac{(1-\alpha_i)c_i e_i}{f_i}. \tag{14}$$

[3]We neglect the downlink interference as the channel state is unavailable when making decisions [13].

2) RSU Computation: We assume that the task Ω_i is offloaded to RSU $j \in \mathcal{R}$ and further migrated to RSU k for computation. The total workload of RSU $k \in \mathcal{R}$ is defined as

$$\xi_k = \sum_{i\in\mathcal{V}}\sum_{j\in\mathcal{R}}\alpha_i C_{i,j}\beta_{i,j,k}c_i e_i. \tag{15}$$

where $k=j$ represents the workload offloaded from vehicle i to RSU k, and $k \neq j$ denotes the workload migrated from RSU j to RSU k.

We assume that RSU k has a First-In-First-Out (FIFO) queue q_k, whose length is represented by Q_k. We assume that the queue is initially empty. Therefore, the length of the queue at next time slot can be expressed as

$$Q'_k = \max\left(0, Q_k + \xi_k - F_k\Delta t\right). \tag{16}$$

where F_k is the computing capacity of the RSU k. For simplicity, the waiting time is estimated based on the queue state at the beginning of the slot. Therefore, the waiting delay for the task Ω_i to be processed at the RSU k can be expressed as

$$T_{i,j,k}^{wait} = \frac{\alpha_i C_{i,j}\beta_{i,j,k}Q_k}{F_k}. \tag{17}$$

Then, the computation delay of the task Ω_i at RSU k is given by

$$T_{i,j,k}^{comp} = \frac{\alpha_i C_{i,j}\beta_{i,j,k}c_i e_i}{F_k}. \tag{18}$$

E. Delay Model

The total delay incurred by task offloading, migration, and computation for the offloaded task Ω_i is formulated as

$$T_i^{off} = \sum_{j\in\mathcal{R}}\sum_{k\in\mathcal{R}}\left(T_{i,j}^{up}+T_{i,j,k}^{mig}+T_{i,j,k}^{wait}+T_{i,j,k}^{comp}\right). \tag{19}$$

Specifically, the task result needs to be transmitted from the computing RSU k to the RSU l via R2R communication, then transmitted from RSU l to vehicle i through V2R downlink communication. Therefore, the total result feedback delay of the task Ω_i is defined as

$$T_i^{back} = \sum_{j\in\mathcal{R}}\sum_{k\in\mathcal{R}}\sum_{l\in\mathcal{R}}\left(T_{i,j,k,l}^{fb}+T_{i,j,k,l}^{down}\right). \tag{20}$$

The task completion delay of the task Ω_i is defined as

$$T_i = T_i^{loc}+T_i^{off}+T_i^{back}. \tag{21}$$

F. Problem Formulation

Based on the above analysis, we aim to minimize the task completion delay by jointly optimizing the task offloading decisions $\mathcal{A}=\{\alpha_i \mid i\in\mathcal{V}\}$, task migration decisions $\mathcal{B}=\{\beta_{i,j,k} \mid i\in\mathcal{V}, j,k\in\mathcal{R}\}$ and channel selection decision $\mathcal{Z}=\{Z_{i,j,u} \mid i\in\mathcal{V}, j\in\mathcal{R}, u\in\mathcal{U}\}$. Specifically, we define the task migration and resource optimization problem in the VEC system as a multi-constraint optimization problem, i.e.,

$$\min_{\mathcal{A},\mathcal{B},\mathcal{Z}} \quad \frac{1}{T_{\max}}\sum_{t=0}^{T_{\max}}\sum_{i=1}^{V}T_i \qquad (\mathbf{P1})$$

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

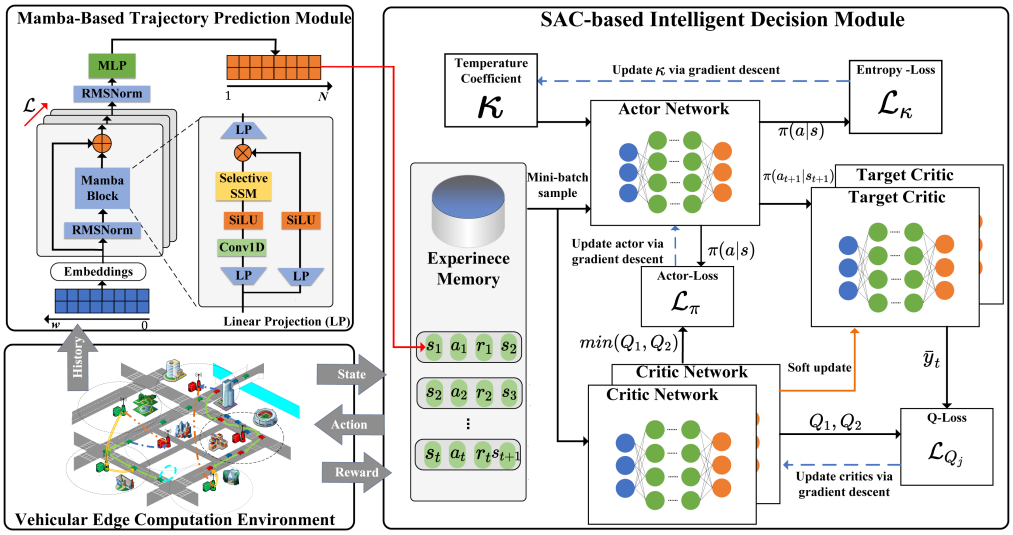


Fig. 2. The architecture of MATM.

$$\text{s.t.} \quad \alpha_i \in \{0,1\}, \forall i \in \mathcal{V}, \tag{22a}$$

$$C_{i,j} \in \{0,1\}, \forall i \in \mathcal{V}, j \in \mathcal{R}, \tag{22b}$$

$$\beta_{i,j,k} \in \{0,1\}, \forall i \in \mathcal{V}, j,k \in \mathcal{R}, \tag{22c}$$

$$Z_{i,j,u} \in \{0,1\}, \forall i \in \mathcal{V}, j \in \mathcal{R}, u \in \mathcal{U}, \tag{22d}$$

$$T_i \leq \tau_i, \forall i \in \mathcal{V}, \tag{22e}$$

$$(3), (4), (5), (7). \tag{22f}$$

Condition (22a)–(22d) represent the value range of task offloading, task migration, and channel selection variables, respectively. Constraint (22e) ensures that the task completion delay does not exceed the maximum allowable delay for each vehicle. (22f) specifies that during offloading, the vehicle must remain associated with only one RSU, tasks can only be offloaded and migrated to one RSU, and the vehicle can communicate using only one subchannel.

The optimization problem $\mathbf{P1}$ belongs to the class of MINLP problems, as it involves both binary and integer decision variables and exhibits nonlinear task delay characteristics [14], [25]. MINLP problems are generally NP-hard, rendering the search for the global optimum of $\mathbf{P1}$ extremely difficult [13]. In addition, the dynamic nature of the environment limits the effectiveness of conventional optimization techniques. By contrast, DRL has demonstrated strong potential to tackle these challenges, providing intelligent decision-making capabilities to efficiently manage non-convex and time-varying problems [34]. We propose an intelligent decision-making algorithm based on mobility-aware DRL.

IV. Mobility-Aware Task Migration

This section first presents a vehicle trajectory prediction module based on Mamba. Subsequently, by integrating an intelligent decision-making module based on SAC, we propose the MATM framework to achieve proactive task migration and resource management.

A. Multi-Level Mamba-Based Trajectory Prediction Module

Accurate trajectory prediction is crucial for dynamic task migration, supporting mobility-aware scheduling and delay reduction [30]. To achieve this, we propose a multi-layer Mamba-based residual module. Formally, the prediction task is to map a historical input sequence $X_i = (L_i^v[t-M+1], \ldots, L_i^v[t])$ of length M to a future sequence $Y_i = (L_i^v[t+1], \ldots, L_i^v[t+N])$ of length N. Here, $L_i^v[t] = (x_i[t], y_i[t])$ denotes the two-dimensional coordinates of vehicle i at time slot t.

Using a sliding window, historical trajectories are batched as the input $\mathbf{X} \in \mathbb{R}^{b \times w \times 2}$, where b indicates the batch size and w denotes the length of the sliding window. To enhance the representational power of the input, we map the raw spatial coordinates and temporal information into a higher-dimensional feature space through a linear embedding layer, i.e.,

$$\mathbf{H}^{(1)} = \text{Embedding}(\mathbf{X}) \in \mathbb{R}^{b \times w \times d}, \tag{23}$$

where d is the hidden layer dimension. As shown in Fig. 2, the embedded input $\mathbf{H}^{(1)}$ is passed through $\mathcal{L}$ Mamba blocks to capture the temporal dynamics of the trajectories. At l-th layer, the output $\mathbf{H}^{(l+1)} \in \mathbb{R}^{b \times w \times d}$ is produced by processing $\mathbf{H}^{(l)} \in \mathbb{R}^{b \times w \times d}$. Specifically, $\mathbf{H}^{(l)}$ is processed through two distinct linear operations $\psi(\cdot)$ and $\psi'(\cdot)$ to expand the input dimension, generating two parallel branches, i.e.,

$$\mathbf{P}^{(l)} = \psi(\mathbf{H}^{(l)}) \in \mathbb{R}^{b \times w \times d'}, \tag{24}$$

$$\mathbf{Q}^{(l)} = \psi'(\mathbf{H}^{(l)}) \in \mathbb{R}^{b \times w \times d'}, \tag{25}$$

where d' represents the expanded dimension. The component $\mathbf{P}^{(l)}$ is processed by a 1-D convolution followed by a SiLU activation, i.e.,

$$\mathbf{P}'^{(l)} = \text{SiLU}\left(\text{Conv1D}(\mathbf{P}^{(l)})\right) \in \mathbb{R}^{b \times w \times d'}. \tag{26}$$

Specifically, perform a linear projection on $\mathbf{P}'^{(l)}$ to obtain the projection matrices B, C, and the step size Δ, while the transition matrix A is generated with fixed parameters. The dynamic timestep Δ is then used to discretize A and B, generating $\overline{A}$

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Multi-level Mamba-based residual trajectory prediction module.

Input: Historical trajectory data X_i.
Output: Predicted future trajectory Y_i.
Initialize Mamba architecture and parameters;
Obtain input $\mathbf{X}$ via sliding window and batch sampling;
Embed $\mathbf{X}$ to obtain $\mathbf{H}^{(1)}$;
/* Multi-level Mamba */
for *level* $l = 1$ *to* $\mathcal{L}$ **do**
  $\mathbf{P}^{(l)} \leftarrow \psi(\mathbf{H}^{(l)})$;
  $\mathbf{Q}^{(l)} \leftarrow \psi'(\mathbf{H}^{(l)})$;
  $\mathbf{P}'^{(l)} \leftarrow \mathrm{SiLU}(\mathrm{Conv1D}(\mathbf{P}^{(l)}))$;
  $A \leftarrow$ Parameter;
  $B \leftarrow$ Linear projection for $\mathbf{P}'^{(l)}$;
  $C \leftarrow$ Linear projection for $\mathbf{P}'^{(l)}$;
  $\Delta \leftarrow \mathrm{Softplus}(\text{Parameter} + \mathrm{Linear}(\mathbf{P}'^{(l)}))$;
  $\overline{A}, \overline{B} \leftarrow \mathrm{Discretize}(\Delta, A, B)$;
  $\mathbf{P}''^{(l)} \leftarrow \mathrm{SelectiveSSM}(\mathbf{P}'^{(l)}, \overline{A}, \overline{B}, C)$;
  $\mathbf{Y}^{(l)} \leftarrow \mathbf{P}''^{(l)} \otimes \mathrm{SiLU}(\mathbf{Q}^{(l)})$;
  /* Residual connection */
  $\mathbf{H}^{(l+1)} \leftarrow \psi''(\mathbf{Y}^{(l)}) + \mathbf{H}^{(l)}$;
end
$Y_i \leftarrow \mathrm{MLP}(\mathbf{H}^{(\mathcal{L}+1)})$;

and $\overline{B}$. The selective SSMs module combines these inputs to generate $\mathbf{P}''^{(l)}$, i.e.,

$$\mathbf{P}''^{(l)} = \text{Selective SSMs}(\mathbf{P}'^{(l)}, \overline{A}, \overline{B}, C) \in \mathbb{R}^{b\times w\times d'}. \quad (27)$$

Finally, $\mathbf{P}''^{(l)}$ is modulated by the component $\mathbf{Q}^{(l)}$ via a SiLU activation gating mechanism, i.e.,

$$\mathbf{Y}^{(l)} = \mathbf{P}''^{(l)} \otimes \mathrm{SiLU}(\mathbf{Q}^{(l)}) \in \mathbb{R}^{b\times w\times d'}. \quad (28)$$

The output of each layer is computed via a linear projection $\psi''(\cdot)$ followed by a residual connection, where the linear projection restores the feature dimension and the residual connection stabilizes training and mitigates vanishing gradients. Thus the output $\mathbf{H}^{(l+1)}$ of layer l is defined as

$$\mathbf{H}^{(l+1)} = \psi''(\mathbf{Y}^{(l)}) + \mathbf{H}^{(l)} \in \mathbb{R}^{b\times w\times d}. \quad (29)$$

The final representation $H^{(\mathcal{L}+1)}$ is passed through a Multi-Layer Perceptron (MLP) to predict the next N timesteps of 2D trajectory coordinates Y_i. Since the key spatial information is the vehicle's RSU assignment, the predicted 2D positions Y_i are converted into RSU assignments for the next N timesteps $\hat{Y}_i[t] = \{\hat{y}_i[t+1], \ldots, \hat{y}_i[t+N]\}$, where $\hat{y}[t]$ denotes the RSU that vehicle i is connected to at time slot t.

The complete workflow of multilayer Mamba-based residual trajectory prediction is shown in Algorithm 1.

B. SAC-Based Intelligent Decision-Making Module

DRL can adaptively learn policies to handle the highly dynamic and uncertain nature of vehicles, RSUs, and channels in VEC scenarios [18]. Integrating a trajectory prediction module enhances state representation and provides reliable decisions for task offloading, migration, and channel selection.

We represent the system described above as an MDP, which is characterized by the components of state, action, and reward.

1) State: The system state at time slot t is defined as

$$s_t = \left\{S_V[t], S_R[t], CSI[t], \hat{Y}[t]\right\}, \quad (30)$$

where $S_V = \{(e_i[t], c_i[t], f_i[t])\}_{i\in\mathcal{V}}$ contains task details and computing capability of vehicles at time slot t, $S_R = \{(Q_j[t], F_j[t])\}_{j\in\mathcal{R}}$ represents the queue lengths and computing capabilities of RSUs, and $CSI = \{H^{up}_{i,j}[t]\}_{i\in\mathcal{V}, j\in\mathcal{R}}$ captures the channel state, reflecting the distance between vehicles and RSUs. In particular, $\hat{Y}[t]$ represents each vehicle's predicted future trajectory positions, expressed as the RSUs to which they are expected to be associated. High-dimensional state features may hinder algorithm convergence and reduce learning efficiency, so we normalize the state of each time slot [23].

2) Action: The executed action of the agent at time slot t is the decision variable required by **P1**, which includes task offloading decisions $\mathcal{A}$, task migration decisions $\mathcal{B}$ and channel selection decision $\mathcal{Z}$, i.e.,

$$\tilde{a}_t = \{\mathcal{A}, \mathcal{B}, \mathcal{Z}\}. \quad (31)$$

Notably, decisions are related to the time slot t.

3) Reward: The reward mainly considers task delay and includes penalties for task failure. Specifically, shorter delay result in higher rewards, while task failures incur a penalty Φ to guide the agent toward more reliable and efficient decision-making. Therefore, the reward at time slot t is defined as:

$$r_t = -\sum_{i\in\mathcal{V}} \left(T_i[t] + Penalty_i\right), \quad (32)$$

$$Penalty_i = \begin{cases} \Phi, & \text{if } T_i[t] > \tau_i, \\ 0, & \text{otherwise.} \end{cases} \quad (33)$$

Popular DRL algorithms such as PPO, TD3, and SAC are widely used. Due to its off-policy nature and entropy-regularized objective, SAC offers improved sample efficiency and better exploration. Therefore, based on the integration of trajectory prediction information, we designed the MATM algorithm incorporating SAC.

SAC adopts an actor-critic architecture with a dual-Q mechanism. The actor network $\mathcal{P}$ generates a policy by producing a distribution over actions, while the critic networks $\mathcal{Q}_1$ and $\mathcal{Q}_2$ evaluate the corresponding Q-values. To stabilize training, target critic networks $\hat{\mathcal{Q}}_1$ and $\hat{\mathcal{Q}}_2$ are used. At each step, the agent observes the VEC environment and integrates the trajectory prediction results to construct the state s_t. Then, the agent generates an action a_t by sampling from the policy $\pi(\cdot|s_t;\theta)$. Specifically, the agent generates raw actions as continuous values within the range $[-1, 1]$. We then apply a continuous-to-discrete mapping technique to convert a_t into discrete $\tilde{a}_t$ that adhere to the constraints in **P1** [35], i.e.,

$$\tilde{a}_t \leftarrow \mathrm{Mapping}(a_t). \quad (34)$$

The discrete action $\tilde{a}_t$ interacts with the environment to acquire r_t and trigger transition to s_{t+1}. Since the reward calculation involves result feedback delays, a single interaction may

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

span multiple time slots. Upon receiving the full reward, the trajectory tuple $e_t = (s_t, a_t, r_t, s_{t+1})$ is stored in the experience replay buffer $\mathcal{D}$ for policy optimization. MATM aims to maximize both the expected reward and the policy entropy to optimize the policy, i.e.,

$$\pi^* = \arg\max_{\pi} \sum_{t} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi} \left[r_t + \kappa \mathcal{H}(\pi(\cdot|s_t)) \right], \tag{35}$$

where κ is the temperature coefficient and $\mathcal{H}(\pi)$ is the entropy of the policy π. ρ_π denotes the distribution of state-action pairs (s_t, a_t) induced by policy π.

The critic networks are updated at each step by minimizing the mean squared error (MSE) between predicted Q-value and the target values. Specifically, the target values are computed as

$$\bar{y}_t = r_t + \gamma \left(\min_{j=1,2} \hat{Q}_j(s_{t+1}, a_{t+1}) - \kappa \log \pi(a_{t+1}|s_{t+1}) \right), \tag{36}$$

where the next action $a_{t+1} \sim \pi(\cdot|s_{t+1})$, γ is the discount factor and the target Q-values $\hat{Q}_j$ are obtained from the target critic networks $\hat{\mathcal{Q}}_j$. The two parameters ϕ_1 and ϕ_2 of critic networks $\mathcal{Q}_1$ and $\mathcal{Q}_2$ are updated by minimizing the MSE loss

$$\mathcal{L}_{Q_j} = \left(Q_j(s_t, a_t) - \bar{y}_t \right)^2, \quad j \in \{1, 2\}. \tag{37}$$

Two critics reduce Q-value bias, while the actor network learns a policy that maximizes returns and encourages exploration. Its parameters θ are updated by minimizing the objective function $\mathcal{L}_\pi$, ensuring a balance between exploiting known rewards and exploring new actions, i.e.,

$$\mathcal{L}_\pi = \left(\kappa \log \pi(a_t|s_t) - \min_{j=1,2} Q_j(s_t, a_t) \right), \tag{38}$$

where the actions a_t are sampled based on current policy using the reparameterization trick. This policy framework strikes a balance between exploration and exploitation.

A key component of MATM is the soft update of the target critics using Polyak averaging, which allows the target networks to smoothly track the evolution of the online critic parameters. Specifically, the parameters $\hat{\phi}_1$ and $\hat{\phi}_2$ of the target networks $\hat{\mathcal{Q}}_1$ and $\hat{\mathcal{Q}}_2$ are updated as

$$\phi'_j \leftarrow \tau \phi_j + (1 - \tau) \phi'_j, \quad j \in \{1, 2\}, \tag{39}$$

where τ is the soft update coefficient that controls how much of the critic network parameters ϕ_j influence the target critic network parameters ϕ'_j.

To ensure an effective trade-off between exploration and exploitation, MATM utilizes a self-adjusting entropy mechanism. Instead of manually tuning the entropy temperature coefficient κ, the algorithm dynamically optimizes κ by minimizing the following objective:

$$\mathcal{L}_\kappa = \mathbb{E}_{a_t\sim\pi} \left[-\kappa \left(\log \pi(a_t|s_t) + \bar{\mathcal{H}} \right) \right], \tag{40}$$

where $\bar{\mathcal{H}}$ is the target entropy that specifies the desired degree of action randomness. The detailed process of MATM is presented in Algorithm 2.

Algorithm 2: MATM for VEC scenario.

Input: VEC environment, pretrained Mamba model.
Output: Optimal service policy $\pi^*(a|s;\theta)$.

Initialize the critic networks $\mathcal{Q}_1, \mathcal{Q}_2$ with ϕ_1, ϕ_2, the target critic networks $\hat{\mathcal{Q}}_1, \hat{\mathcal{Q}}_2$ with $\hat{\phi}_1, \hat{\phi}_2$, and the actor network $\mathcal{P}$ with θ;
Initialize the replay buffer $\mathcal{D}$ and κ;
for *each episode* **do**
  Initialize the initial state s_1 of the environment;
  for $t = 1$ *to* T_{max} **do**
    Predict the vehicle trajectories via Algorithm 1;
    $s_t \leftarrow$ update state with predicted trajectories;
    Sample an action a_t based on the state s_t using the policy $\pi(a|s_t;\theta)$;
    $\tilde{a}_t \leftarrow \text{Mapping}(a_t)$;
    Execute the selected action $\tilde{a}_t$;
    Observe the reward r_t and the next state s_{t+1};
    Store the experience (s_t, a_t, r_t, s_{t+1}) into $\mathcal{D}$;
    if $\mathcal{D}$ *is full* **then**
      Randomly sample a mini-batch (s_j, a_j, r_j, s_{j+1}) from the replay buffer $\mathcal{D}$;
      Update critic networks $\mathcal{Q}_1, \mathcal{Q}_2$ via (37);
      Update actor network $\mathcal{P}$ via (38);
      Update the temperature coefficient via (40);
      for $j = 1, 2$ **do**
        $\phi'_j \leftarrow \tau\phi_j + (1-\tau)\phi'_j$;
      end
    end
  end
end

C. Algorithm Complexity Analysis

The computation complexity of Algorithm 1 is $\mathcal{O}(\mathcal{L}Md^2 + Nd^2)$, where M is the length of the historical trajectory, N is the number of future prediction steps, d is the hidden dimension, and $\mathcal{L}$ is the number of layers.

The computational complexity of Algorithm 2 depends on both the SAC and Algorithm 1. For the SAC module, let the actor network and critic network have L_A and L_C layers, respectively, where the number of neurons in the I-th layer of the actor network and the J-th layer of the critic network are denoted as $\mathcal{F}_I$ and $\mathcal{G}_J$. Thus, the per-step training complexity of the SAC module can be expressed as $\mathcal{O}(\sum_{I=1}^{L_A} \mathcal{F}_I \mathcal{F}_{I+1} + \sum_{J=1}^{L_C} \mathcal{G}_J \mathcal{G}_{J+1})$, the overall time complexity of Algorithm 2 can be expressed as

$$\mathcal{O}\left(ET_{\max} \left(\sum_{I=1}^{L_A} \mathcal{F}_I \mathcal{F}_{I+1} + \sum_{J=1}^{L_C} \mathcal{G}_J \mathcal{G}_{J+1} + V(\mathcal{L}Md^2 + Nd^2) \right) \right), \tag{41}$$

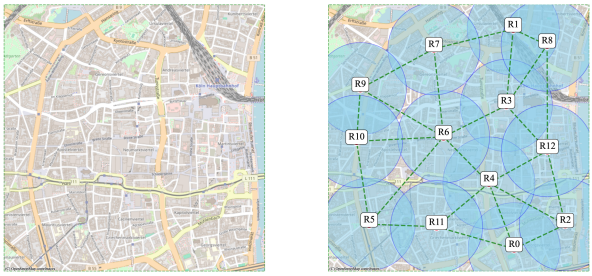
where E is the number of training episodes.

V. Experiments and Results

A. Experiment Settings

1) Experiment scenarios: We adopt a real-world vehicular trajectory dataset from Cologne, Germany, the data sampling interval is 1 s [36]. As illustrated in Fig. 3(a), we select the

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 3. The actual road of Cologne city center. (a) Urban area of Cologne. (b) RSUs deployment map.

TABLE II
TRAINING PARAMETER SETTINGS

Module	Symbol	Parameter	Value
SAC	lr_a	Actor learning rate	1×10^{-5}
	lr_c	Critic learning rate	1×10^{-5}
	B_m	Mini-batch size	128
	κ	Temperature coefficient	0.02
	$\mathcal{D}$	Replay buffer size	20000
	τ	Soft update coefficient	0.005
	E	Total training episodes	5000
Mamba	w	Input dimensions	32
	N	Output dimensions	8
	$\mathcal{L}$	Number of stacked layers	4
	d	Hidden dimension per layer	64
	b	Batch size	128

central 2 km $\times$ 2 km urban area of Cologne as the experimental area. This area better reflects high-load scenarios and enables more effective evaluation of the proposed MATM. As shown in Fig. 3(b), based on the dataset-provided RSU locations, we deploy 13 RSUs within this area. The coverage radius of RSU is set to 350 m. To better resemble realistic topologies, we construct a minimum spanning tree to ensure basic connectivity among RSUs. Then, we apply a K-nearest neighbors strategy with $K = 3$ to further link RSUs, enhancing connectivity and redundancy and forming a more realistic and robust RSU communication graph. Regarding the RSU deployment, if a vehicle is located within the coverage areas of multiple RSUs, it selects the nearest RSU as its serving node. Furthermore, if a vehicle is located outside the coverage area of any RSU, it opts for local computing instead.

2) Parameter settings: Experiments were conducted using the PyTorch 2.1.0 framework, with hyperparameters empirically tuned for optimal performance.

Table II lists the training parameters for the SAC and Mamba modules. Following the parameter guidelines in [30], we pre-trained the Mamba model for 5 epochs on a 24-hour dataset to ensure learning stability and prediction reliability. Table III summarizes key environmental parameters. The R2R communication rate is set to 500 Mbit/s [30], the propagation rate of electromagnetic waves is assumed to 2×10^5 km/s, and the system employs 5 subchannels. We uniformly sample the computational capability of RSU j within [10, 30] GHz as the default in experiments. Similarly, we uniformly sample each

TABLE III
ENVIRONMENT PARAMETER SETTINGS

Symbol	Description	Default Value
V	Number of vehicles	50
R	Number of RSUs	13
F_j	Computing capacity of RSU j	[10, 30] GHz
f_i	Computing capacity of vehicle i [23]	[1, 3] GHz
e_i	Task data size of vehicle i	[10, 15] Mb
μ_i	Ratio between result/input data size	0.5
c_i	Cycles required to process one bit data	[500, 1000] cycles/bit
W	V2R communication bandwidth [3], [7]	20 MHz
P_i	Transmission power of vehicle i [34]	0.1 W
g_0	Channel power gain at 1m [34]	-47 dB
σ_0^2	Noise power [7]	-96 dBm
T_{max}	The total system time slots	60
Δt	The duration of each time slot [37]	1s

vehicle's computing capacity, task data size, and CPU cycles per bit within allowable ranges.

3) Baselines: We conduct comparative experiments with heuristic algorithms and state-of-the-art DRL approaches to evaluate the contribution of MATM:

- TD3-based Solution (**TD3S**) [8]: TD3 is a value-based DRL method that improves stability by using two Q-networks, delayed updates, and target smoothing.
- PPO-based Solution (**PPOS**) [23]: PPO is a policy gradient algorithm that promotes stable training by clipping policy updates, preventing sudden and excessive changes.
- **Static**: Vehicles are statically assigned to fixed RSUs to ensure a balanced workload distribution across all RSUs.
- **Random**: Tasks are offloaded to RSUs randomly without any specific optimization strategy.

4) Evaluation Metrics: In the comparative and ablation experiments, we used the following metrics:

- Average Task Delay (**ATD**): The average task completion delay per vehicle per step, calculated as

$$\text{ATD} = \frac{1}{E \cdot T_{\max} \cdot V} \sum_{e=1}^{E} \sum_{t=1}^{T_{\max}} \sum_{i=1}^{V} T_i[t], \tag{42}$$

- Task Success Rate (**TSR**): The average ratio of tasks successfully completed within deadlines

$$\text{TSR} = \frac{1}{E \cdot T_{\max} \cdot V} \sum_{e=1}^{E} \sum_{t=1}^{T_{\max}} \sum_{i=1}^{V} \mathbb{I}\left(T_i[t] - \tau_i \leq 0\right), \tag{43}$$

where $\mathbb{I}(\cdot)$ represents an indicator function that outputs 1 if the enclosed condition is satisfied.

- RSU Workload Intensity (**RWI**): The workload intensity of RSU j at time slot t is defined as

$$\text{RWI}_j[t] = \frac{Q_j[t] + \xi_j[t]}{F_j[t]}. \tag{44}$$

- Load Balancing Metric (**LBM**): Measures the standard deviation of RSU workloads to reflect workload balancing performance.

$$\text{LBM} = \frac{1}{E \cdot T_{\max}} \sum_{e=1}^{E} \sum_{t=1}^{T_{\max}} \sqrt{\frac{1}{R} \sum\nolimits_{j \in \mathcal{R}} \left(\text{RWI}_j[t] - \overline{\text{RWI}}[t]\right)^2}, \tag{45}$$

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

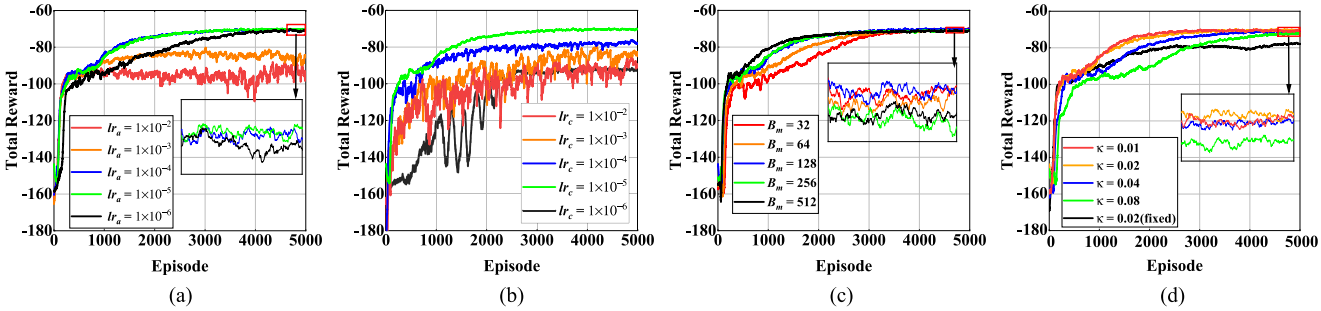


Fig. 4. Training performance under different parameter settings. (a) Actor learning rates. (b) Critic learning rates. (c) Batch sizes. (d) Temperature coefficients.

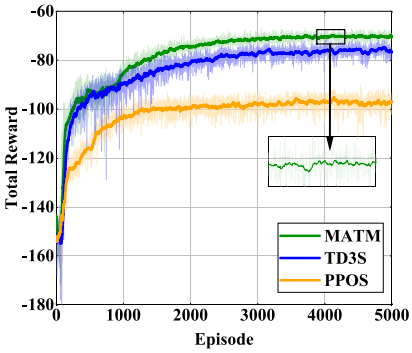


Fig. 5. Comparison of rewards for MATM, TD3S, and PPOS.

where $\overline{\mathrm{RWI}}[t]$ represents the average RWI of all RSUs at time slot t.

- Result Feedback Delay (**RFD**): The average delay for transmitting results from RSUs to vehicles, calculated as

$$\mathrm{RFD} = \frac{1}{E \cdot T_{\max} \cdot V} \sum_{e=1}^{E} \sum_{t=1}^{T_{\max}} \sum_{i=1}^{V} T_i^{back}[t]. \tag{46}$$

B. Convergence Performance Comparison

We evaluate the impact of key parameters on MATM's convergence performance, with results shown in Fig. 4. Specifically, Fig. 4(a) and (b) demonstrate that MATM achieves optimal convergence when lr_a and lr_c are set to 1×10^{-5}. Furthermore, Fig. 4(c) shows that MATM attains the best final convergence with batch size $B_m = 128$. We compare the convergence performance of MATM under different initial values of κ, with Fig. 4(d) indicating that MATM achieves the best convergence performance when $\kappa = 0.02$. Notably, the adaptive temperature adjustment outperforms the fixed $\kappa = 0.02$ in convergence speed and final reward, verifying the effectiveness of dynamic temperature adjustment.

As shown in Fig. 5, we compared the convergence performance of MATM, PPOS, and TD3S under identical experimental settings. MATM converges at around 2800 episodes and achieves higher rewards, while TD3S converges at approximately 3000 episodes. Although PPOS converges relatively faster and exhibits more stable training behavior, it tends to get stuck in local optima. Overall, MATM demonstrates superior performance by providing efficient and stable convergence along with robustness against local optima. MATM's entropy-enhanced objective is the main factor behind the accelerated convergence, as it provides an effective trade-off between exploration and exploitation, especially in challenging DRL scenarios. In fact, as depicted in the zoomed inset of Fig. 5, the reward values in each episode may exhibit considerable fluctuations due to the stochastic nature of the environment. In comparison, MATM exhibits significantly smaller fluctuations in the later training stages, demonstrating enhanced robustness and stability.

C. Performance Comparison and Parameter Analysis

To evaluate each algorithm's performance, we conducted 100 tests under the default experimental settings[4] and five different scenarios.

1) Default Experimental Settings: Table IV compares the performance of different methods under the default experimental settings. MATM achieves the best overall results across Total Reward, ATD, TSR, and RFD. Although MATM achieves 0.3% improvement in TSR compared to TD3S, it reduces the key metric ATD by 8.3% and outperforms TD3S on the remaining metrics as well. PPOS shows good load-balancing performance but performs worse than MATM and TD3S in terms of ATD and TSR, likely due to convergence to local optima. The Static method attains the best load balance through uniform task allocation but suffers from significantly higher ATD and RFD and a lower TSR because it lacks dynamic scheduling. Random performs the worst on all metrics. Overall, MATM demonstrates the highest scheduling efficiency and robustness, reducing the primary metric ATD by 8.3%–54.5% compared to the baseline methods.

2) Number of Vehicles: As shown in Fig. 6, we evaluate the impact of different numbers of vehicles on system performance. Specifically, Fig. 6(a) shows that ATD increases as the number of vehicles grows, mainly due to intensified competition for computing and communication resources. As shown in Fig. 6(b), communication resources remain adequate for the number of vehicles ranging from 30 to 40, as indicated by a 1.2% decrease in the rate. However, when the number of vehicles increases from 50 to 60, the transmission rate drops significantly by 11.5%, owing to excessive competition for limited channel resources. Fig. 6(d) shows that the LBM gradually worsens with increasing

[4]The default experimental settings correspond to the parameter configurations listed in Table III.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PERFORMANCE COMPARISON OF DIFFERENT METHODS WITH DEFAULT EXPERIMENTAL SETTINGS

Methods	Total Reward	ATD (s)	LBM	TSR	RFD (s)
MATM	**-70.912 ± 1.891**⋆	**1.229 ± 0.039**⋆	0.4572 ± 0.0171	**96.12% ± 0.82%**⋆	**0.1359 ± 0.0092**⋆
TD3S	-76.536 ± 2.001	1.340 ± 0.045	0.5918 ± 0.0183	95.82% ± 1.51%	0.1361 ± 0.0098
PPOS	-96.809 ± 2.344	1.612 ± 0.052	0.4989 ± 0.0173	93.33% ± 1.66%	0.1366 ± 0.0111
Static	-151.283 ± 4.521	2.142 ± 0.066	**0.1410 ± 0.0097**⋆	82.42% ± 3.25%	0.1421 ± 0.0332
Random	-197.430 ± 18.417	2.704 ± 0.159	1.1189 ± 0.1642	75.10% ± 6.02%	0.1523 ± 0.0437

⋆ indicates the optimal value of a single metric.

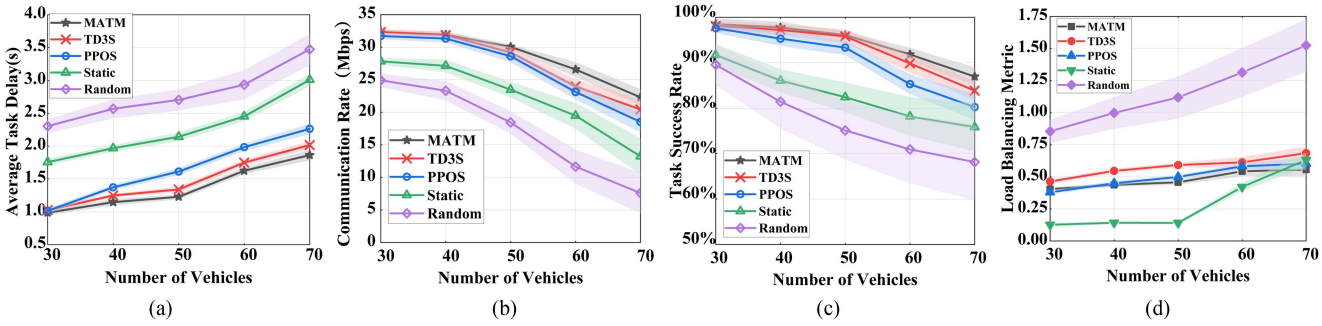


Fig. 6. Performance of various metrics under different numbers of vehicles. (a) ATD. (b) V2R Uplink Communication Rate. (c) TSR. (d) LBM.

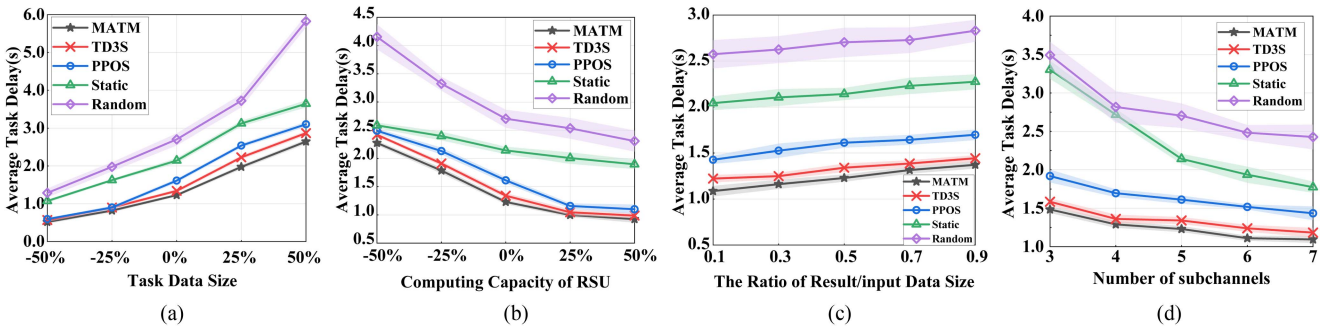


Fig. 7. Performance of ATD under different parameter settings. (a) Task Data Size. (b) Computing Capacity of RSU. (c) The Ratio of Result/Input Data Size. (d) Number of subchannels.

vehicle numbers, indicating growing imbalance and insufficiency in RSU computing resource allocation. Even under such competitive conditions, MATM consistently achieves the best ATD and TSR performance, demonstrating superior stability and robustness against interference.

3) Task Data Size: As shown in Fig. 7(a), we evaluate the impact of task sizes on ATD, focusing on how RSU workload affects overall behavior, while task size has little effect on channel interference. In this experiment, task sizes are uniformly sampled from [10, 15] Mb as the default value, corresponding to $\pm$ 50% deviations from the default value [8]. The results indicate that heavier computational workloads augment the processing burden of RSUs, leading to queue accumulation and significant delay. This overloading effect is particularly pronounced under the Random offloading strategy. Static strategies aim to maximize RSU utilization and reduce queue buildup but show relatively minor changes in ATD due to their lack of adaptability. In contrast, DRL-based approaches such as MATM, PPOS, and TD3S dynamically adjust scheduling policies, effectively mitigating performance degradation under high load. Overall, MATM demonstrates superior robustness and efficiency, highlighting its effectiveness in managing heavily loaded systems.

4) Computing Capacity of RSUs: As shown in Fig. 7(b), we evaluate the variation in ATD under different RSU computing capacities, with capacities scaled from -50% to $+50\%$ relative to the default setting, sampled from a uniform distribution of [10, 30] GHz. The results indicate that ATD generally decreases as RSU computing capacity increases. When RSU computing capacity is reduced, the unchanged task demands from vehicles may exceed the available RSU resources, causing more vehicles to resort to local computation and leading to increased delay. Compared to other baseline methods, MATM demonstrates stronger adaptability to variations in RSU computing capacity and consistently maintains near-optimal performance under different conditions.

5) The Ratio of result/input Data Size: As shown in Fig. 7(c), we evaluate the impact of different ratios of result/input data size ranging from 0.1 to 0.9 on ATD. As the ratio of result/input increases, the system must handle a larger volume of task return data, leading to higher ATD. Since the Random strategy

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

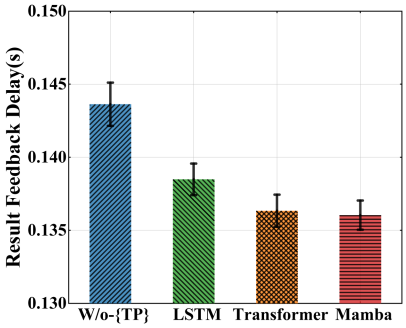


Fig. 8. Comparison of RFD under different trajectory prediction methods.

lacks adaptive scheduling logic, it offloads fewer tasks to RSUs compared to other methods. Its ATD is relatively insensitive to changes in the task result return ratio, showing minor delay variation. In contrast, for other methods, ATD rises noticeably with increasing task return volume, indicating that higher task return data significantly affects system delay. By dynamically adjusting scheduling policies, MATM clearly maintains lower ATD across different ratios of result/input data size, demonstrating superior stability and robustness.

6) Number of Subchannels: As shown in Fig. 7(d), we investigate the impact of varying the number of subchannels on system performance. This highlights the importance of comparing different methods under varying subchannel conditions, as it reveals how each approach handles interference and resource contention. Although SIC is employed to mitigate interference, interference intensifies as the number of subchannels decreases. This leads to increased ATD due to prolonged transmission delay. However, as the number of subchannels increases, the system gains sufficient parallel capacity to support concurrent vehicle transmissions. Nevertheless, since the total bandwidth is evenly divided among subchannels, an excessive number of channels may decrease per-channel bandwidth. This trade-off explains why the delay from 6 to 7 subchannels does not continue to decrease. Furthermore, under varying subchannel configurations, MATM consistently outperforms PPOS and TD3S in terms of communication efficiency.

D. Ablation Experiments

In this section, we conduct ablation experiments to analyze the impact of the task migration module and the trajectory prediction module on system performance metrics. We compare MATM with the following baseline variants:

- Full Offloading (**FO**): All tasks are executed at RSUs, effectively ignoring the intelligent decision-making module.
- Without Task Migration (**W/o-{TM}**): Tasks are only offloaded to the current associated RSU or executed locally, with decisions optimized via SAC.
- Without Trajectory Prediction (**W/o-{TP}**): The task migration module relies solely on the SAC framework, without access to predicted vehicle trajectories.

1) Impact of the Trajectory Prediction Module: As shown in Fig. 8, to highlight the impact of the trajectory prediction module on RFD, we evaluate the RFD under different trajectory prediction methods and compared the results with W/o-{TP}. This experiment focuses on analyzing the impact of different methods on RFD, which reflects their prediction performance. The results show that incorporating a trajectory prediction module reduces RFD by 3.6%–5.2% compared to W/o-{TP}. Specifically, the Mamba-based model outperforms LSTM and Transformer by reducing RFD by 1.8% and 0.2%, respectively. Although Transformer and Mamba achieve similar reductions in RFD, as shown in Table V, Mamba exhibits an 11.7% shorter inference time than the Transformer, indicating better real-time performance. Overall, the trajectory prediction module simplifies the task feedback process and reduces RFD, with the Mamba-based implementation achieving the most significant improvement.

TABLE V
INFERENCE TIME COMPARISON PER VEHICLE PER STEP FOR VARIOUS TRAJECTORY PREDICTION METHODS

Model	LSTM	Mamba	Transformer
Number of Layers	4	4	4
Neurons per Layer	64	64	64
Inference Time (ms)	3.553	3.781	4.282

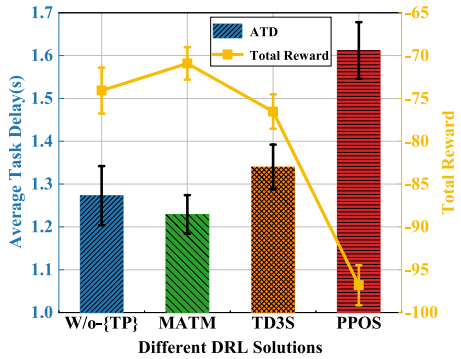


Fig. 9. Comparison of Total Reward and ATD under DRL methods.

As shown in Fig. 9, we compare the total reward and ATD of W/o-{TP} with other DRL methods. The results indicate that MATM exhibits a 4.3% higher total reward and a 3.5% lower ATD than W/o-{TP}. This demonstrates that the trajectory prediction module effectively enhances the decision-making capability of the SAC framework. Meanwhile, W/o-{TP} still outperforms TD3S and PPOS, indicating that the SAC architecture itself provides stronger policy advantages. Therefore, the performance improvement of MATM arises from the combined effect of the SAC structure and the trajectory prediction module. Fig. 10 shows that W/o-{TP} starts to exhibit obvious performance degradation and fluctuations at around 2500 episodes, while MATM converges steadily. This indicates that the trajectory prediction module can improve DRL stability by providing foresight guidance in dynamic environments.

2) Impact of Task Migration Module: As shown in Fig. 11, to highlight the role of the task migration module in achieving workload balancing, we compare the average RWI per minute across three methods: FO, W/o-{TM}, and MATM. In the FO approach, all tasks are offloaded directly to RSUs, resulting

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

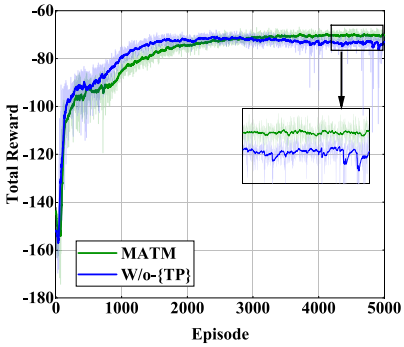


Fig. 10. Comparison of rewards for MATM and W/o-{TP}.



Fig. 11. Comparison of RWI under different methods.

in severe queuing, overload, and significant workload imbalance among RSUs. In the W/o-{TM} method, where the task migration module is removed, offloading decisions rely solely on the SAC-based decision framework. While this approach can partially optimize task offloading, the workload offloaded to RSUs is significantly reduced, and RSU resources are not fully utilized, leading to suboptimal workload balancing. By integrating MATM with the task migration module, the system enables effective collaboration among RSUs. Task migration distributes the RWI more evenly, alleviating individual RSU overload and allowing as many tasks as possible to be executed without overburdening any specific RSU.

VI. Conclusion

This paper proposes a mobile-aware task migration framework (MATM) that integrates spatio-temporal vehicle information with trajectory prediction and DRL-based intelligent decision-making. MATM facilitates collaborative task migration and resource allocation for vehicles in VEC scenario. Experiments conducted in real urban scenarios demonstrate that MATM outperforms baseline approaches in terms of ATD and TSR, providing an efficient and intelligent method for task migration and resource management.

Future work will focus on extending the framework to heterogeneous computing nodes and large-scale task migration scenarios. We will strive to improve trajectory prediction accuracy and integrate distributed trajectory prediction with multi-agent DRL-based decision-making mechanisms.

References

[1] H. Zhou, K. Jiang, S. He, G. Min, and J. Wu, "Distributed deep multi-agent reinforcement learning for cooperative edge caching in Internet-of-Vehicles," *IEEE Trans. Wireless Commun.*, vol. 22, no. 12, pp. 9595–9609, Dec. 2023.

[2] Y. Hou, Z. Wei, R. Zhang, X. Cheng, and L. Yang, "Hierarchical task offloading for vehicular fog computing based on multi-agent deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 23, no. 4, pp. 3074–3085, Apr. 2024.

[3] X. Dai, Z. Xiao, H. Jiang, and J. C. S. Lui, "UAV-Assisted task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2520–2534, Apr. 2024.

[4] K. Zhang, J. Cao, and Y. Zhang, "Adaptive digital twin and multiagent deep reinforcement learning for vehicular edge computing and networks," *IEEE Trans. Ind. Inform.*, vol. 18, no. 2, pp. 1405–1413, Feb. 2022.

[5] Z. Sun, G. Sun, Y. Liu, J. Wang, and D. Cao, "BARGAIN-MATCH: A game theoretical approach for resource allocation and task offloading in vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1655–1673, Feb. 2024.

[6] C. Zhang et al., "Distributed deep reinforcement learning-based gradient quantization for federated learning enabled vehicle edge computing," *IEEE Internet Things J.*, vol. 12, no. 5, pp. 4899–4913, Mar. 2025.

[7] J. Chi, X. Zhou, F. Xiao, Y. Lim, and T. Qiu, "Task offloading via prioritized experience-based double dueling DQN in edge-assisted IIoT," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14575–14591, Dec. 2024.

[8] W. Fan, Y. Zhang, G. Zhou, and Y. Liu, "Deep reinforcement learning-based task offloading for vehicular edge computing with flexible RSU-RSU cooperation," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7712–7725, Jul. 2024.

[9] Y. Wu, X. Fang, G. Min, H. Chen, and C. Luo, "Intelligent offloading balance for vehicular edge computing and networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 5, pp. 5792–5803, May 2025.

[10] Y. Shen, B. Li, R. Zhang, X. Cheng, and L. Yang, "A flexible load balancing scheme in multi-UAV-enabled wireless networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 9205–9210, Jun. 2024.

[11] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, and M. Xiao, "Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15513–15526, Dec. 2023.

[12] J. Yang, Q. Yuan, S. Chen, H. He, X. Jiang, and X. Tan, "Cooperative task offloading for mobile edge computing based on multi-agent deep reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3205–3219, Sep. 2023.

[13] Z. Nan, S. Zhou, Y. Jia, and Z. Niu, "Joint task offloading and resource allocation for vehicular edge computing with result feedback delay," *IEEE Trans. Wireless Commun.*, vol. 22, no. 10, pp. 6547–6561, Oct. 2023.

[14] H. Huang, W. Zhan, G. Min, Z. Duan, and K. Peng, "Mobility-aware computation offloading with load balancing in smart city networks using MEC federation," *IEEE Trans. Mobile Comput.*, vol. 23, no. 11, pp. 10411–10428, Nov. 2024.

[15] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," in *Proc. 1st Conf. Lang. Model.*, Jul. 2024, pp. 1–32.

[16] Y. Ren, L. Liu, Z. Lan, Z. Cui, and H. Yu, "MLB-Traj: Map-free trajectory prediction with local behavior query for autonomous driving," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 28556–28571, Jul. 2025.

[17] Z. Lan, L. Liu, B. Fan, Y. Lv, Y. Ren, and Z. Cui, "Traj-LLM: A new exploration for empowering trajectory prediction with pre-trained large language models," *IEEE Trans. Intell. Veh.*, vol. 10, no. 2, pp. 794–807, Feb. 2025.

[18] L. Zhao et al., "A digital twin-assisted intelligent partial offloading approach for vehicular edge computing," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3386–3400, Nov. 2023.

[19] A. Rabee and I. Barhumi, "Toward energy-efficient dynamic resource allocation in uplink NOMA systems: Deep reinforcement learning for single and multi-cell NOMA systems," *IEEE Trans. Veh. Technol.*, vol. 74, no. 6, pp. 9313–9327, Jun. 2025.

[20] P. Dai, K. Hu, X. Wu, H. Xing, F. Teng, and Z. Yu, "A probabilistic approach for cooperative computation offloading in MEC-assisted vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 899–911, Feb. 2022.

[21] H. Wang, T. Lv, Z. Lin, and J. Zeng, "Energy-delay minimization of task migration based on game theory in MEC-assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8175–8188, Aug. 2022.

[22] J. Li, Q. Jiang, V. C. M. Leung, Z. Ma, and K. K. Abrokwa, "Deep-reinforcement-learning-based joint optimization of task migration and resource allocation for mobile-edge computing," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 24431–24440, Jul. 2025.

[23] Q. Chen, X. Song, T. Song, and Y. Yang, "Vehicular edge computing networks optimization via DRL-based communication resource allocation and load balancing," *IEEE Trans. Mobile Comput.*, vol. 24, no. 9, pp. 9222–9237, Sep. 2025.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.

[24] C. Liu, F. Tang, Y. Hu, K. Li, Z. Tang, and K. Li, "Distributed task migration optimization in MEC by extending multi-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1603–1614, Jul. 2021.
[25] Y. Li, L. Li, and P. Fan, "Mobility-aware computation offloading and resource allocation for NOMA MEC in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 8, pp. 11934–11948, Aug. 2024.
[26] I. Maity, R. Dhiman, and S. Misra, "MobiPlace: Mobility-aware controller placement in software-defined vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 957–966, Jan. 2021.
[27] W. Zhao, P. Gao, X. Hong, X. Zheng, and N. Kato, "PPO based task offloading with EKF for position prediction in RSU-assisted IoV," *IEEE Trans. Cogn. Commun. Netw.*, vol. 12, pp. 703–713, 2026, doi: 10.1109/TCCN.2025.3561209.
[28] J. Chen et al., "Multiagent deep reinforcement learning for dynamic avatar migration in AIoT-enabled vehicular metaverses with trajectory prediction," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 70–83, Jan. 2024.
[29] C.-L. Wu, T.-C. Chiu, C.-Y. Wang, and A.-C. Pang, "Mobility-aware deep reinforcement learning with seq2seq mobility prediction for offloading and allocation in edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 6803–6819, Jun. 2024.
[30] X. Zhang, C. Wang, Y. Zhu, J. Cao, and T. Liu, "Multi-agent deep reinforcement learning with trajectory prediction for task migration-assisted computation offloading," *IEEE Trans. Mobile Comput.*, vol. 24, no. 7, pp. 5839–5856, Jul. 2025.
[31] Y. Huang, Y. Cheng, and K. Wang, "Trajectory mamba: Efficient attention-Mamba forecasting model based on selective SSM," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2025, pp. 12058–120 67.
[32] S. Cheng, S. Das, S. Qu, and L. Ballan, "KD-Mamba: Selective state space models with knowledge distillation for trajectory prediction," *Comput. Vis. Image Understand.*, vol. 261, Nov. 2025, Art. no. 104499.
[33] Z. Xiao, H. Hu, G. Xu, and J. He, "TAME: Temporal audio-based Mamba for enhanced drone trajectory estimation and classification," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, Apr. 2025, pp. 1–5.
[34] P. Hou, H. Zhu, Z. Lu, S.-C. Huang, Y. Yang, and H. Chai, "Learning-based over-the-air integrated sensing, communication and computation in UAV swarm-enabled intelligent transportation systems," *IEEE Trans. Green Commun. Netw.*, vol. 9, no. 3, pp. 1414–1428, Sep. 2025.
[35] Q. Liu, H. Zhang, X. Zhang, and D. Yuan, "Improved DDPG based two-timescale multi-dimensional resource allocation for multi-access edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 9153–9158, Jun. 2024.
[36] S. Uppoor, O. Trullols-Cruces, M. Fiore, and J. M. Barcelo-Ordinas, "Generation and analysis of a large-scale urban vehicular mobility dataset," *IEEE Trans. Mobile Comput.*, vol. 13, no. 5, pp. 1061–1075, May 2014.
[37] J. Huang, M. Zhang, J. Wan, Y. Chen, and N. Zhang, "Joint data caching and computation offloading in UAV-assisted Internet of Vehicles via federated deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 17644–17656, Nov. 2024.



Yilin Yin received the B.S. degree from the Shandong University of Science and Technology, Qingdao, China, in 2024. He is currently working toward the M.S. degree with the School of Future Science and Engineering, Soochow University, Suzhou, China. His research interests include mobile edge computing, reinforcement learning, and sensing fusion.



Peng Hou (Member, IEEE) received the B.S. degree in communication engineering from the University of Science and Technology Beijing, Beijing, China, in 2018, the M.S. degree in electronic and communication engineering from Yunnan University, Kunming, China, in 2021, and the Ph.D. degree in cyberspace security from Fudan University, Shanghai, China, in 2025. He is currently a Lecturer with the School of Future Science and Engineering, Soochow University, Suzhou, China. His research interests including edge intelligence, reinforcement learning, Integrated Sensing and Communications (ISAC) and UAV networks.



Shihong Cui is currently working toward the Ph.D. degree in computer science and artificial intelligence with the Wuhan University of Technology, Wuhan, China. He is the General Manager with Marketing Department, Tianyijiaotong Technology Company Ltd. His research interests include C-V2X, 5G networks, intelligent connected vehicles, and smart transportation.



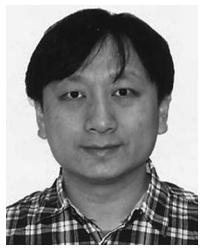
Jin Huang received the B.E. and Ph.D. degrees from the College of Mechanical and Vehicle Engineering, Hunan University, Changsha, China, in 2006 and 2012, respectively. From 2009 to 2011, he was a joint Ph.D. with the College of Mechanical and Vehicle Engineering, Hunan University, and the George W. Woodruff School of Mechanical Engineering, Georgia Institute of Technology, Atlanta, GA, USA. In 2013, he joined Tsinghua University, Beijing, China, as a Postdoc, where he has been an Assistant Research Professor, since 2016. His research interests include artificial intelligence in intelligent transportation systems, dynamics control, and fuzzy engineering.



Jin Wang (Member, IEEE) received the B.S. degree from the Ocean University of China, Qingdao, China, in 2006, and the Ph.D. degree in computer science jointly awarded by the City University of Hong Kong, Hong Kong, and University of Science and Technology of China, Hefei, China, in 2011. He is currently a Professor with the School of Future Science and Engineering, Soochow University, Suzhou, China. His research interests include collaboration and interaction of autonomous intelligent systems, resource scheduling in computing power network, and AI+robotics.



Zhihui Lu (Member, IEEE) received the Ph.D. degree computer science degree from Fudan University, Shanghai, China, in 2004. He is currently a Professor with the School of Computer Science, Fudan University. He has coauthored two books and more than 100 journal articles and conference papers in his research interests which include cloud computing and service computing technology, Big Data architecture, edge computing, and IoT distributed systems. He is a member of IEEE and China Computer Federation's Service Computing Specialized Committee.



Patrick C. K. Hung (Senior Member, IEEE) received the Ph.D. degree from the School of Computer Science, Hong Kong University of Science and Technology, Hong Kong, in 2011. He is currently a Professor with the Faculty of Business and Information Technology, Ontario Tech University, Oshawa, ON, Canada. He has been with Boeing Research and Technology, Seattle, WA, USA, on aviation services-related research projects. His research interests include services computing, cloud computing, blockchain, Big Data, and edge computing.

Authorized licensed use limited to: Goa University. Downloaded on October 26,2026 at 09:14:31 UTC from IEEE Xplore. Restrictions apply.